

Q9: Feature Matching under Illumination Changes

Course: Computer Vision (ITA05) | Assessment Tool 2: Case Study | CO3

Scenario

Two images of the same scene are captured under different lighting conditions, causing feature matching errors. Analyze why matching fails and explain how preprocessing or robust descriptors improve performance.

Analysis

Feature detectors like SIFT/ORB locate keypoints at locations of strong, stable intensity gradients (corners, blobs, textured regions). A large, non-uniform illumination change -- for example a directional lighting gradient combined with overall darkening (gamma shift) -- alters the local gradient magnitude at many locations. Regions that were high-contrast under normal lighting can become very low-contrast (near black) under the new lighting, so the detector simply fails to find a keypoint there at all.

In the experiment, the well-lit scene produced 172 SIFT keypoints, but the same scene under a strong gamma + directional-gradient illumination change produced only 48 keypoints -- many real corners were lost in the darkened region. Even where keypoints are still detected, the descriptor (which encodes local gradient orientation histograms) is influenced by the change in local contrast, reducing the descriptor similarity between the two images and lowering the good-match count.

The result is fewer correspondences overall, and a lower fraction of correct matches, which directly hurts downstream tasks such as image stitching, object recognition, or visual tracking.

Suggested Solution / Improvements

- Normalize illumination before feature detection using CLAHE (Contrast Limited Adaptive Histogram Equalization) applied to the L channel of the LAB colour space. Unlike global histogram equalization, CLAHE equalizes contrast locally in small tiles and clips the histogram to avoid over-amplifying noise, which restores gradient structure in both bright and dark regions.
- Use descriptors that are inherently more robust to illumination change. SIFT descriptors are normalized (unit-length gradient histograms) which already gives some invariance to linear/affine brightness changes; ORB and other binary descriptors based on local intensity comparisons are also relatively robust, since they compare relative rather than absolute intensities.
- Combine both: apply CLAHE first, then extract SIFT/ORB keypoints and descriptors and match with a ratio test as usual.

Output / Result Screenshot

Q9: Feature Matching Under Illumination Changes (Raw vs CLAHE-Preprocessed)

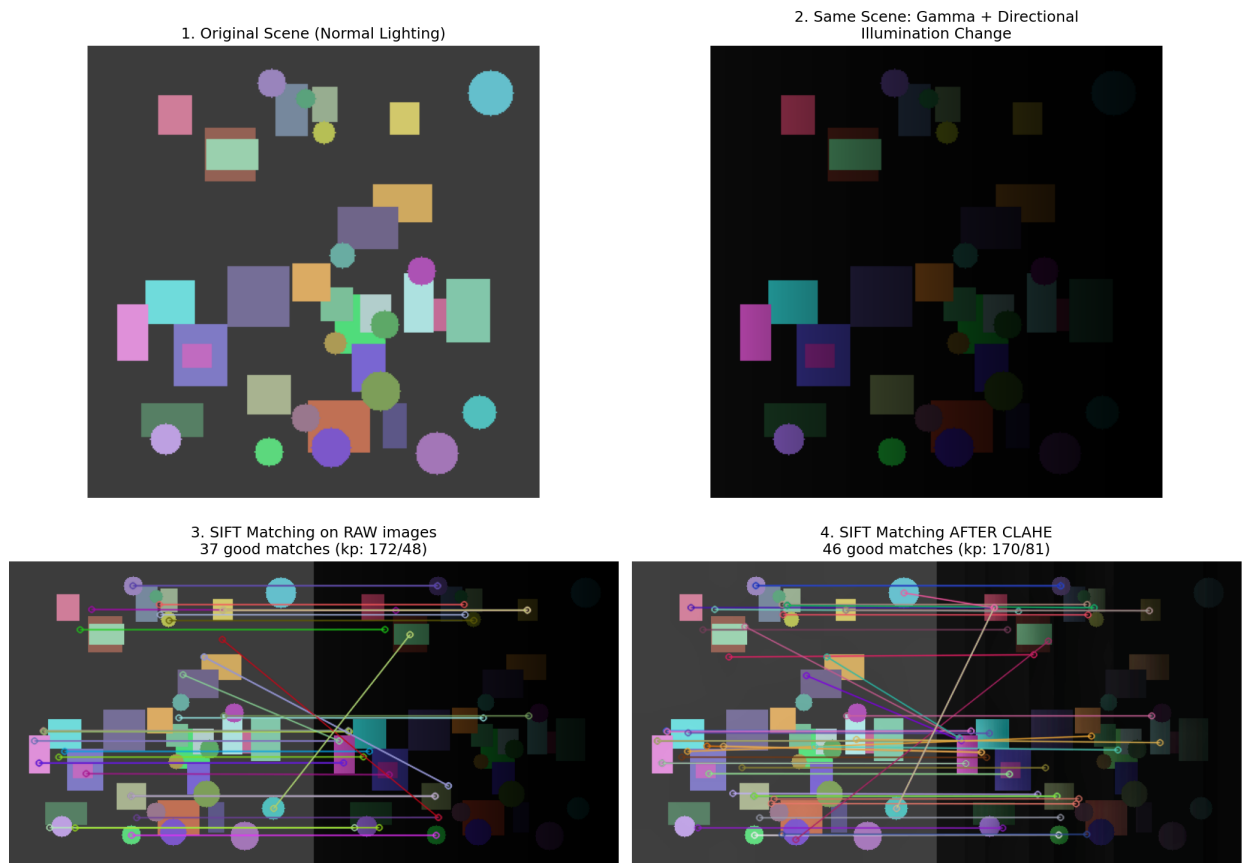


Figure: Output generated by Q9_illumination_feature_matching.py

Conclusion

After applying CLAHE to both the well-lit and the darkened image, the keypoint count in the darkened image rose from 48 to 81 and the number of good matches after the ratio test increased from 37 to 46, confirming that illumination-normalization preprocessing directly improves feature matching robustness under lighting changes.

Implementation (Python / OpenCV)

Full source code is provided separately as **Q9_illumination_feature_matching.py**. The key implementation is reproduced below.

```
"""
Question 9: Feature Matching under Illumination Changes
-----
Two images of the same scene are captured under different lighting
conditions, causing feature matching errors. This script shows why
raw matching struggles and how preprocessing (CLAHE - histogram
equalization) plus robust descriptors (SIFT) restore accuracy.

Pipeline:
1. Build a synthetic textured "scene".
2. Create a second version with a strong, non-uniform illumination
   change (darkened + gamma shift + directional lighting gradient).
3. Match SIFT descriptors on the RAW (unequalized) images -> fewer /
   weaker good matches because gradients/intensities differ.
4. Apply CLAHE (Contrast Limited Adaptive Histogram Equalization) to
   normalize illumination, then match again -> more, stronger matches.
5. Save a comparison figure as the output screenshot.
"""

import cv2
import numpy as np
import matplotlib.pyplot as plt

def build_scene(size=320):
    """Synthetic scene with rich texture/corners for feature detection."""
    img = np.zeros((size, size, 3), dtype=np.uint8)
    img[:] = (60, 60, 60)
    rng = np.random.default_rng(3)

    for _ in range(25):
        x, y = rng.integers(20, size - 60, size=2)
        w, h = rng.integers(15, 45, size=2)
        color = tuple(int(c) for c in rng.integers(80, 230, size=3))
        cv2.rectangle(img, (x, y), (x + w, y + h), color, -1)

    for _ in range(15):
        x, y = rng.integers(20, size - 20, size=2)
        r = int(rng.integers(6, 18))
        color = tuple(int(c) for c in rng.integers(80, 230, size=3))
        cv2.circle(img, (x, y), r, color, -1)

    return img

def apply_illumination_change(img):
    """Simulate a strong lighting change: darker overall + gradient + gamma."""
    h, w = img.shape[:2]
    # Directional lighting gradient (dark on right, brighter on left)
    gradient = np.linspace(1.0, 0.25, w).reshape(1, w, 1)
    lit = img.astype(np.float32) * gradient

    # Overall darkening (gamma correction)
    gamma = 2.2
    lit = 255.0 * (lit / 255.0) ** gamma
    lit = np.clip(lit, 0, 255).astype(np.uint8)
    return lit

def match_sift(img1, img2, label):
    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(gray1, None)
    kp2, des2 = sift.detectAndCompute(gray2, None)

    if des1 is None or des2 is None or len(des1) < 2 or len(des2) < 2:
        return None, 0, 0, 0

    bf = cv2.BFMatcher()
    matches = bf.knnMatch(des1, des2, k=2)
    good = [m for m, n in matches if m.distance < 0.75 * n.distance]

    match_img = cv2.drawMatches(img1, kp1, img2, kp2, good, None,
                                flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    print(f"{label}: kp1={len(kp1)}, kp2={len(kp2)}, good_matches={len(good)}")
    return match_img, len(kp1), len(kp2), len(good)

def clahe_normalize(img):
    """Apply CLAHE on the L channel of LAB color space to normalize illumination."""
```

```

lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
l_eq = clahe.apply(l)
lab_eq = cv2.merge((l_eq, a, b))
return cv2.cvtColor(lab_eq, cv2.COLOR_LAB2BGR)

def main():
    scene = build_scene()
    dark_scene = apply_illumination_change(scene)

    raw_match_img, kp1_raw, kp2_raw, good_raw = match_sift(scene, dark_scene, "RAW (no preprocessing)")

    scene_eq = clahe_normalize(scene)
    dark_eq = clahe_normalize(dark_scene)
    eq_match_img, kp1_eq, kp2_eq, good_eq = match_sift(scene_eq, dark_eq, "CLAHE-preprocessed")

    fig, axes = plt.subplots(2, 2, figsize=(14, 11))

    axes[0, 0].imshow(cv2.cvtColor(scene, cv2.COLOR_BGR2RGB))
    axes[0, 0].set_title('1. Original Scene (Normal Lighting)')
    axes[0, 0].axis('off')

    axes[0, 1].imshow(cv2.cvtColor(dark_scene, cv2.COLOR_BGR2RGB))
    axes[0, 1].set_title('2. Same Scene: Gamma + Directional\nIllumination Change')
    axes[0, 1].axis('off')

    axes[1, 0].imshow(cv2.cvtColor(raw_match_img, cv2.COLOR_BGR2RGB))
    axes[1, 0].set_title(f'3. SIFT Matching on RAW images\n{good_raw} good matches (kp: {kp1_raw}/{kp2_raw})')
    axes[1, 0].axis('off')

    axes[1, 1].imshow(cv2.cvtColor(eq_match_img, cv2.COLOR_BGR2RGB))
    axes[1, 1].set_title(f'4. SIFT Matching AFTER CLAHE\n{good_eq} good matches (kp: {kp1_eq}/{kp2_eq})')
    axes[1, 1].axis('off')

    plt.suptitle('Q9: Feature Matching Under Illumination Changes (Raw vs CLAHE-Preprocessed)',
                 fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.savefig('/home/claude/cv_solutions/images/Q9_output.png', dpi=150, bbox_inches='tight')
    print("Saved output screenshot to Q9_output.png")

if __name__ == "__main__":
    main()

```